

Code-breaking Neural Networks

Tristan Hodgson

January 5, 2026

1 Motivation

This project aims to replicate and extend previous work in applying machine learning to cryptanalysis, particularly the work of Sam Greydanus [2], where a recurrent neural network was used to decrypt Enigma ciphertext given the key. We extend this work by considering different ciphers and explore how known prefixes can accelerate the learning process.

2 Data

All data is gathered from the Project Gutenberg [1] corpus of public-domain books. Text is preprocessed by converting to lowercase and removing all non-alphabetic characters to standardise the input space to 26 characters (plus padding, beginning-of-string, and end-of-string tokens).

We train using pairs of

$$(\text{key} + f(\text{prefix} + \text{text}), \text{key} + \text{prefix} + \text{text}),$$

where f is the chosen cipher applied to the plaintext.

We consider cases where

$$\text{prefix} = \begin{cases} "" \\ \text{"the quick brown fox jumped over the lazy dog "} \end{cases}.$$

This prefix contains every letter of the alphabet at least once. When encrypted with a substitution or Caesar cipher, the prefix reveals the entire substitution table to the network after it has already seen the key.

The ciphers considered in this project are the Caesar cipher, and the substitution cipher.

We differ slightly from the approach of Greydanus in two main ways. First, we use longer text sequences of 300 words (compared to 14), providing more context at the cost of increased training time and fewer feasible epochs. Second, we introduce a known prefix, which we hypothesise will increase test accuracy by providing additional information about the encryption algorithm.

3 Baseline

Each string has a length of 300 words from the Project Gutenberg dataset. If we include the prefix this goes up to 309 however if the string is longer than 1500 characters we cut it off at or pad to this point to give us constant size tensors.

If we assume that we randomly guess characters (ignoring special tokens) with a uniform distribution we get that we would expect $\frac{25}{26} \times 1500 \approx 1440$ errors. While this calculation is very rough (we could improve it by considering frequency of letters in the dataset) it serves as a useful baseline to see if our model is doing anything useful.

I consider a string of text that has fewer than 100 errors to be decrypted since this roughly means an average of an error every third word. This is an arbitrary boundary but likely a useful one for seeing how well we are really doing.

If we assume our random model we may treat each of the 1500 characters as a Bernoulli distribution with parameter $\frac{25}{26}$. We then sum up all of these Bernoulli distributions and if we get a number less than 100 then this corresponds to a decrypted string. This means that we want the probability that a Binomial with parameters $P(B(1500, \frac{25}{26})) \leq 100 \approx 3 \times 10^{-1825}$. Multiplying by the number of test examples (20000) gives us our expected number of such fluke cases as $\approx 6 \times 10^{-1821}$, in other words, its not happening.

4 Model Architecture

The model is an LSTM-based recurrent neural network, replicating the work demonstrated by Greydanus [2]. Training is supervised: the model receives ciphertext sequences and must predict the corresponding plaintext tokens. At each timestep, the model predicts the next plaintext character until the end-of-string token is generated.

Parameter	Value	Comment
Batch size	32	Lower than Greydanus recommendation of 300 to reduce training time
Embedding dim	256	
Hidden dim	512	
Dropout	0.5	
Epochs	15	
Learning rate	1×10^{-3}	
Betas	(0.9, 0.999)	
Clip	1.0	

5 Results

5.1 Caesar Cipher

The model achieved high accuracy on the Caesar cipher even without the prefix, as the key space is small (26 possible shifts). However, the addition of the prefix further stabilised the learning process.

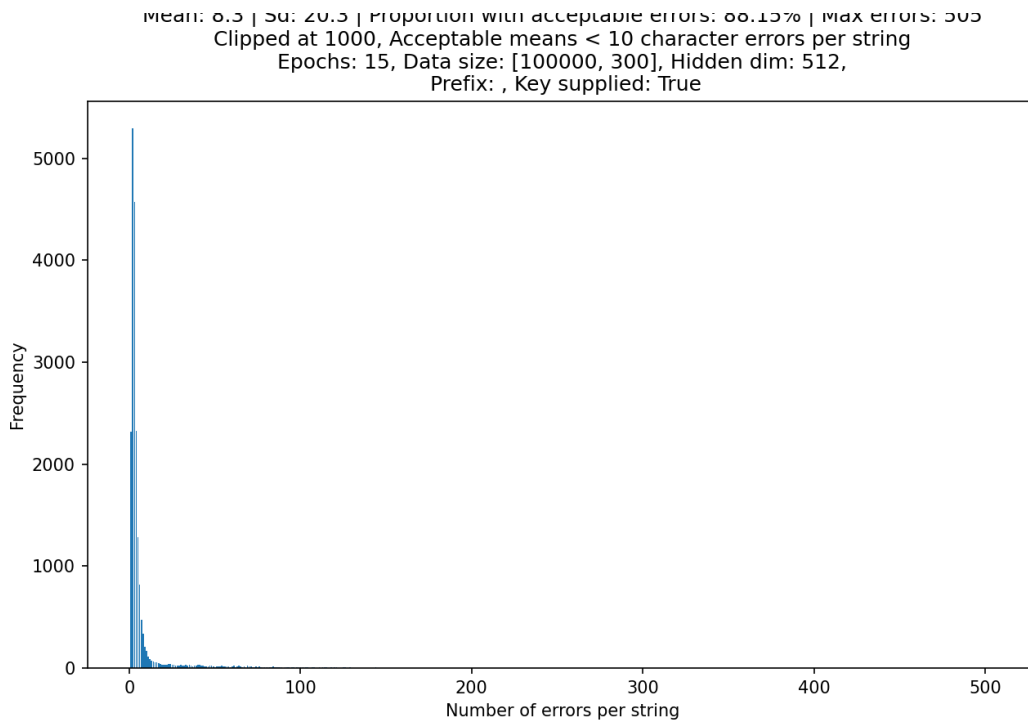


Figure 1: Distribution of errors per string for the Caesar cipher (no prefix).

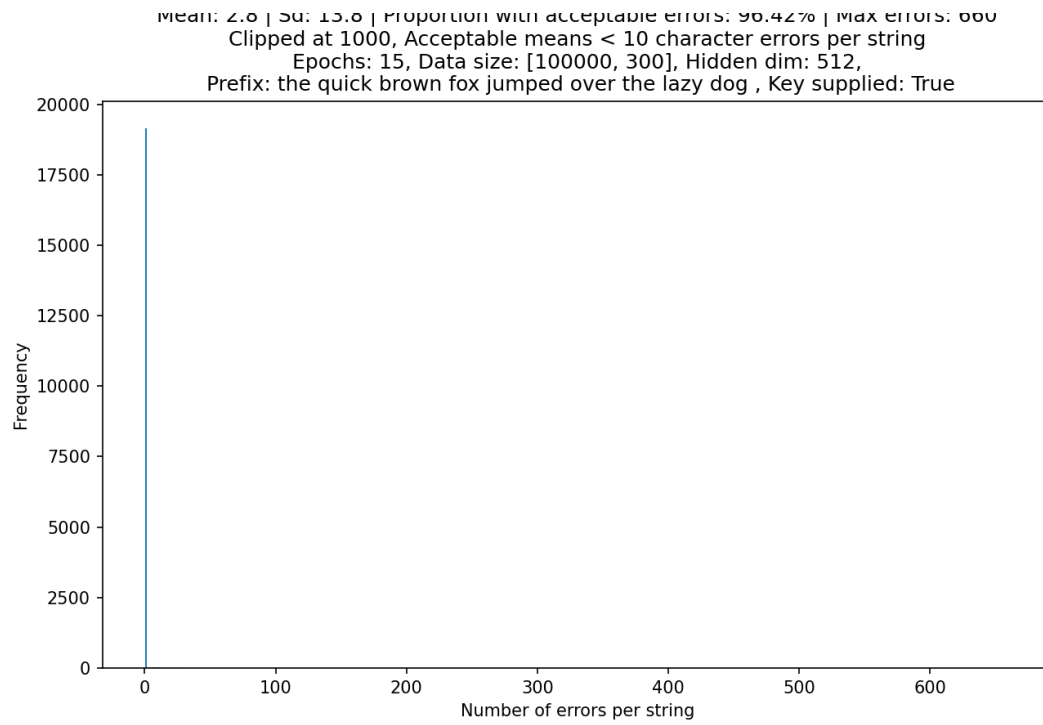


Figure 2: Distribution of errors per string for the Caesar cipher (with prefix). Note the reduction in outliers.

5.2 Substitution Cipher

The substitution cipher presents a much larger key space ($26!$). Without the prefix, the model struggles to infer the full mapping from short sequences alone. However, with the prefix we observe a significant improvement in accuracy.

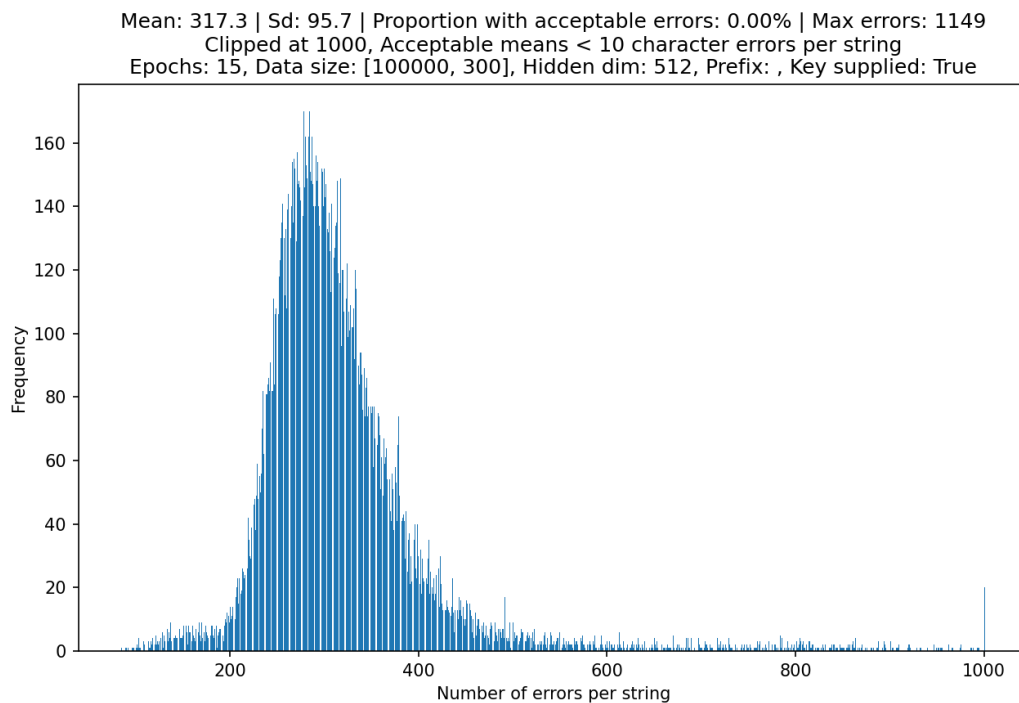


Figure 3: Distribution of errors per string for the substitution cipher (no prefix). The model struggles to recover the full key.

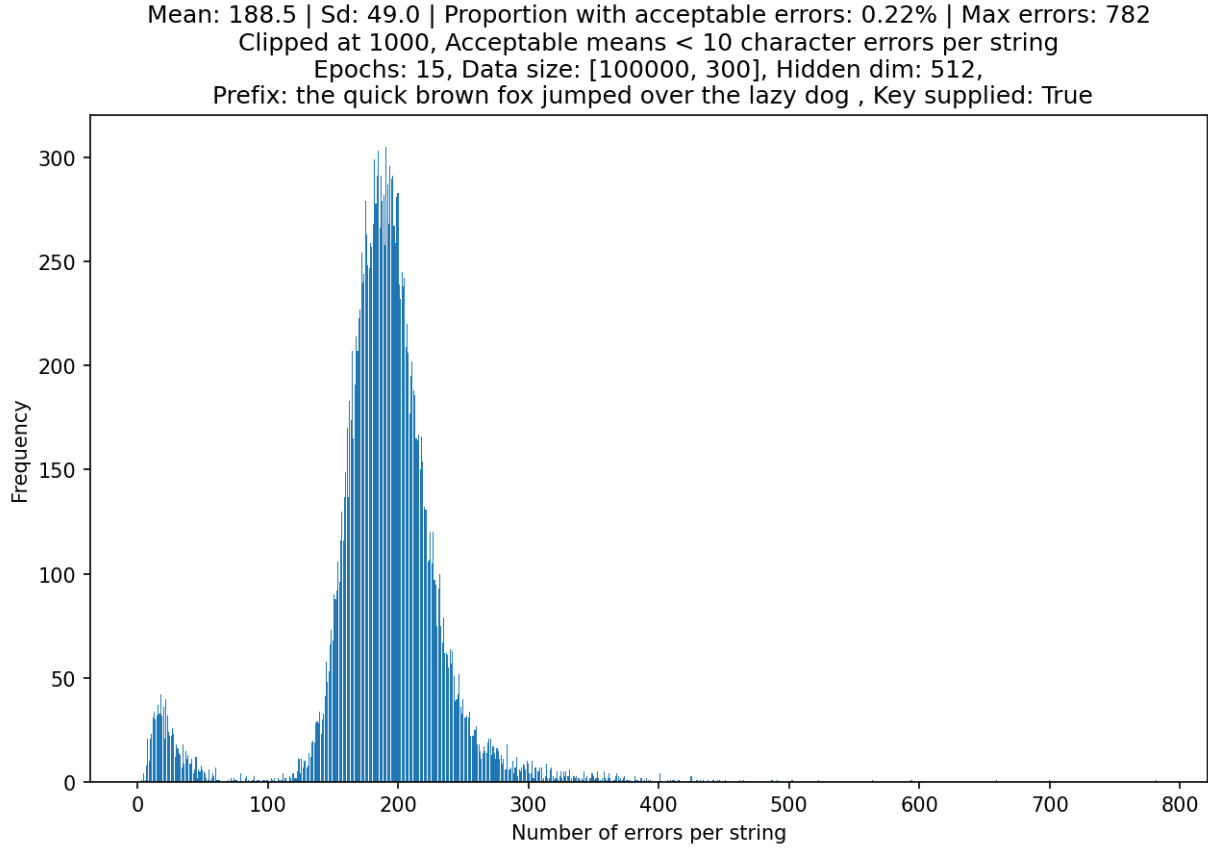


Figure 4: Distribution of errors per string for the substitution cipher (with prefix). The known-plaintext attack allows for significantly better decryption.

6 Future Work

Several extensions could build on this proof of concept:

- **Language Generalisation:** Training the model on different languages (e.g., German) to determine whether the learned decryption algorithms are language-dependent or capture cipher logic independent of linguistic statistics.
- **Mixed Ciphers:** Increasing complexity by randomly switching between two different ciphers (e.g., Caesar and substitution) for each character, forcing the model to infer the encryption method dynamically.
- **Key Reconstruction:** Modifying the architecture to output the encryption key itself rather than plaintext, testing the model’s ability to perform explicit key recovery.

References

- [1] Project gutenber. <https://www.gutenberg.org/>. Accessed 2025-11-22.
- [2] Sam Greydanus. Learning the enigma with recurrent neural networks. *arXiv preprint arXiv:1708.07576*, 2017.